K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering & Technology

Program- B.Tech (AI/ML)

Course Title- Data Structures

Course Code- ETCCDS202

Unit–3 Assignment: Sorting Algorithms

Student Name- Adriana Aharyan Roll no.2501730153

Submission Date-04.03.2026 Section- A

1. Introduction

Sorting is a fundamental operation in computer science used to arrange data in a specific order (ascending or descending). Different sorting algorithms perform differently depending on input size and data distribution.

In this assignment, three sorting algorithms were implemented:

- Insertion Sort
- Merge Sort
- Quick Sort

Their performance was analyzed using different datasets.

◇ 2. Objectives

- To implement sorting algorithms from scratch
- To compare their performance experimentally
- To understand time complexity differences
- To analyze behavior on different input types

◇ 3. Algorithms Implemented

3.1 Insertion Sort

- Works by inserting elements into the correct position
- Time Complexity:
  - Best Case: $O(n)$
  - Worst Case: $O(n^2)$
- Stable and in-place

3.2 Merge Sort

- Uses divide and conquer approach
- Splits list and merges sorted halves
- Time Complexity:
  - Best/Worst Case: $O(n \log n)$
- Stable but not in-place (uses extra memory)

3.3 Quick Sort

- Uses partitioning with a pivot

- Recursively sorts subarrays
- Time Complexity:
  - Average Case: O(n log n)
  - Worst Case: $O(n^2)$
- In-place but not stable

◇ 4. Methodology

4.1 Dataset Generation

Three types of datasets were generated:

- Random data
- Already sorted data
- Reverse sorted data

Sizes used:

- 1000
- 5000
- 10000

4.2 Performance Measurement

- Execution time measured using Python time module
- Same dataset copied before sorting
- Time recorded in milliseconds

◇ 5. Results

Size	Data Type	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
1000	Random	19.37	1.48	1.23
1000	Sorted	0.14	1.11	2.33
1000	Reverse Sorted	37.87	1.20	1.17
5000	Random	556.46	10.38	7.63
5000	Sorted	0.70	6.97	10.07
5000	Reverse Sorted	1094.43	6.22	8.74
10000	Random	2183.20	23.14	15.84
10000	Sorted	0.99	14.30	16.75
10000	Reverse Sorted	4382.21	16.34	17.41

◇ 6. Analysis

6.1 Random Data

- Merge Sort and Quick Sort performed much faster than Insertion Sort
- This matches theoretical complexity:
  - Insertion → $O(n^2)$
  - Merge/Quick → $O(n \log n)$

6.2 Sorted Data

- Insertion Sort performed very efficiently (best case $O(n)$)
- Quick Sort initially showed poor performance with last-element pivot

- After using random pivot, performance improved
- Merge Sort remained consistent

6.3 Reverse Sorted Data

- Insertion Sort showed worst-case behavior ($O(n^2)$)
- Quick Sort also slowed down without pivot optimization
- Merge Sort again remained stable

◇ 7. Theory vs Practical Results

The experimental results match theoretical expectations:

- $O(n^2)$ algorithms (Insertion Sort) become very slow for large inputs
- $O(n \log n)$ algorithms (Merge, Quick) scale efficiently
- Input type significantly affects performance

◇ 8. Special Case: Quick Sort

Initially, Quick Sort used the last element as pivot, which caused:

- Worst-case $O(n^2)$ time
- Deep recursion
- RecursionError for sorted datasets

Solution Applied:

- Random pivot selection was implemented
- This improved performance and avoided worst-case behavior

◇ 9. Stability and Memory Usage

Algorithm	Stable	In-Place	Extra Space
Insertion Sort	Yes	Yes	No
Merge Sort	Yes	No	O(n)
Quick Sort	No	Yes	O(log n) recursion

◇ 10. Conclusion

- Merge Sort is the most consistent algorithm across all inputs
- Quick Sort is fastest on average but depends on pivot choice
- Insertion Sort is efficient only for small or nearly sorted datasets
- Experimental results strongly support theoretical time complexities

---